CSC-223 Data Structures and Analysis of Algorithms Real-life Application

Chuck Cartledge, PhD

February 8, 2026

Contents

1 Introduction **1**

2 Discussion **1**

3 Control file specifications **1**

3.1 System control 3
3.2 Creating data 4
3.3 Retrieving data 5
3.4 Updating data 7
3.5 Deleting data 8

4 Conclusion **8**

A Sample Java listings **9**

B High Level GitHub Instructions **22**

List of Tables

1 CRM system commands. 3
2 **C**reation data commands. 4
3 **R**etieve commands. 5
4 **U**pdating data commands. 7
5 **D**elete commands. 8
6 Sample Java listings. 9

Listings

A.1 Classic: Hello World in Java. 10
A.2 Echo command line arguments to the screen. 10
A.3 Read lines from file, one at a time. 10
A.4 How to base64 encode a string. 11
A.5 How to decode a base64 string. 12
A.6 Create a sample control file. 13
A.7 A sample control file. 18
A.8 Persisting Java data. 20

1 Introduction

Developing a real-life application is a part of the Virginia Community College System (VCCS) mandated requirements for CSC-223 Data Structures and Analysis of Algorithms. This report describes a control file that will be used to create, populate, and query a simple persistent client relationship management (**CRM**) system. How the control file is created is outside the scope of this report.

The control file is a simple ASCII text file populated with assorted creation, retrieval, update, and deletion (**CRUD**) database directives. CRUD related tokens are in plain text, while data associated with the tokens may be in plain text, or base64[1] encoded strings.

2 Discussion

The purpose of this project is to introduce and work with various real-world tools in a safe and instructive manner. The project will be to create the back end of a persistent client relationship management (**CRM**) system. For our purposes:

- The application software will be written in Java,
- The CRM database will be persistent between program executions,
- A GitHub repository will serve as the base application,
- Program teams will make team specific branches from the GitHub base,
- Team members will use local git repositories for local and team versions,
- It will be assumed that a totally different application will interact with the user to create the control data file that is used to give commands to the CRM.

Other details will be fleshed out as the semester progresses.

3 Control file specifications

These are the general specifications of the CRM control file. Detailed specifications can be found in the subsections 3.1 through 3.5. Section 3.1 deals with managing the "system." Sections 3.2 through 3.5 address creating, retrieving, updating, and deleting (**CRUD**) operations. A sample control file created by Listing A.6 is available in Listing A.7 (see Section A).

[1] Base64 is a binary-to-text encoding that uses 64 printable characters to represent each 6-bit segment of a sequence of byte values[3].

1. The control file is a simple ASCII file. It could be created with a text editor like Notepad, gedit, nano, or TextEdit but need not be.
2. Almost all controls will consist of at least three lines. The purpose of the lines are:
   (a) Indicate the start of a something.
   (b) Usually, a base64 encoded string that is used by the "something."
   (c) Indicate the end of something.
3. Lines whose first non white-space character is a # will be treated as a comment, and ignored.
4. A control file will be processed until the **Exit** control is encountered, or the end of file is reached.
5. Control tokens are case sensitive.

The name of the control file will be passed to the CRM as a command line argument.

Not all file names are treated the same across all operating systems.

Different operating systems have different rules when it comes to what constitutes a valid filename. This includes things like:

- Capitalization,
- Length,
- Valid characters, and
- How to navigate from the current working directory to access the file.

For the purposes of this CRM, apply the keep it simple silly (**KISS**) principle and keep all control files in the same working directory where the CRM program is executing.

Not all base64 encoders work the same.

All encoders return a base64 representation of the source characters. A base64 encoded string may span multiple lines. But the length of the returned lines may vary. The only way to ensure a complete base64 line is read, is to read and append lines until an **control ending marker** is read.

3.1 System control

The CRM will be able to run in many different directories, persistently store data in different database files, and manage its data (see Table 1).

Table 1: CRM system commands. A list of system level commands, their beginning and ending markers, and associated base64 encoded data.

Starting Marker	**Plain Text Data**	**Ending Marker**
<System>	A group marker to delimit where data to create or manage a database begins and ends. Only system commands are recognized in between these markers.	< /System>
<Comment>	A plain text message that is output to standard out when processed.	< /Comment>
<DBFileName>	The name of the file where the database is persisted.	< /DBFileName>
<Delete>	The name of the database file to be deleted.	< /Delete>
<Exit>	Terminate the current CRM process. If the field is not **NULL** then output the termination message.	< /Exit>
<Output>	Output retrieved CRM records, default to all records.	< /Output>
<Save>	Save the current database to where it should be persisted. If the field is NULL save to the current database file.	< /Save>
<Sort>	Field used to sort CRM records. There may be more than one Sort directive. All sorts are cumulative.	< /Sort>
<Trace>	Turn trace commands "ON" or "OFF" to track CRM operations.	< /Trace>

3.2 Creating data

Data must be "created" in a database for it to be of much use (see Table 2). Creating a CRM data record will result in a unique identifier that can be used by the other **CRUD** operations. Duplicate entries will result in different identifiers.

Table 2: **C**reation data commands. A list of data level commands, their beginning and ending markers, and associated base64 encoded data.

Starting Marker	**Base64 Encoded Data**	**Ending Marker**
<Create>	A group marker to delimit where data to create a CRM record begins and ends. Only **C**reation commands are recognized in between these markers.	< /Create>
<Comment>	A plain text message that is output to standard out when processed.	< /Comment>
<GivenName>	Given name may be NULL, ASCII, or contain non-ASCII characters.	< /GivenName>
<PostCode>	Post code may be NULL, ASCII, or contain non-ASCII characters.	< /PostCode>
<Surname>	Family name may be NULL, ASCII, or contain non-ASCII characters.	< /Surname>
<Telephone>	Phone number may be NULL, ASCII, or contain non-ASCII characters.	< /Telephone>

3.3 Retrieving data

Data must be "retrieved" from a database for it to be of much use (see Table 3). In this context "retrieved" means that a record is identified for further processing (e.g., sorting, outputting, deleting, etc.). If the identifier is valid, then the entire record is returned, otherwise an error message is returned. Each field (if used), is used to "search" the current database for records that are an exact match for the field. Successive fields will refine the search based on the previously retrieved records. In this way, the entire database is searched the first time, then a subset is searched, then yet another subset is searched, and so on until all the search fields are processed. At the end of the processing, 0 or more CRMIDs are returned that match the search criteria.

Table 3: **R**etieve commands. A list of system level commands, their beginning and ending markers, and associated base64 encoded data. Only **R**etrieve commands are recognized in between these markers.

Starting Marker	**Mixed base64 and plain text Data**	**Ending Marker**
<Retrieve>	A group marker to delimit where data to retrieve a CRM record begins and ends. Only **R**etrieve commands are recognized in between these markers.	< /Retrieve>
<Comment>	A plain text message that is output to standard out when processed.	< /Comment>
<CRMID>	The identifier as returned from the Creation operation. CRMID is always plain text.	< /CRMID>
<GivenName>	Given name may be NULL, ASCII, or contain non-ASCII characters.	< /GivenName>
<PostCode>	Post code may be NULL, ASCII, or contain non-ASCII characters.	< /PostCode>
<Surname>	Family name may be NULL, ASCII, or contain non-ASCII characters.	< /Surname>
<Telephone>	Phone number may be NULL, ASCII, or contain non-ASCII characters.	< /Telephone>

Reserved CRMIDs

There are some CRMIDs that are reserved for special uses. They are case sensitive and have special meaning. They include:

All : Refers to all records in the database.

NULL : Treated the same as **ALL**.

The CRMID **NULL** deserves special attention. **NULL** is the empty string (i.e. a string with 0 characters), vice the 4 character string consisting of the token "NULL". It is recommended that **NULL** not be used as a CRMID to avoid confusion.

3.4 Updating data

Data in a database is dynamic and must be changed from time to time for it to be of much use (see Table 4).

Table 4: **U**pdating data commands. A list of update commands, their beginning and ending markers, and associated base64 encoded data.

Starting Marker	**Base64 Encoded Data**	**Ending Marker**
<Update>	A group marker to delimit where data to update a CRM record begins and ends. Only **U**pdate commands are recognized in between these markers.	< /Update>
<Comment>	A plain text message that is output to standard out when processed.	< /Comment>
<CRMID>	The identifier as returned from the Creation operation. CRMID is always plain text. (The same commands used to create a record are used to update the record.)	< /CRMID>
<GivenName>	Given name may be NULL, ASCII, or contain non-ASCII characters.	< /GivenName>
<PostCode>	Post code may be NULL, ASCII, or contain non-ASCII characters.	< /PostCode>
<Surname>	Family name may be NULL, ASCII, or contain non-ASCII characters.	< /Surname>
<Telephone>	Phone number may be NULL, ASCII, or contain non-ASCII characters.	< /Telephone>

3.5 Deleting data

Data in a database must be "deleted" from a database for it to be of much use (see Table 5). A CRM record can be selected for deletion by an explicit CRMID, or by identifying CRM records that match the search criteria given between the <Delete> and < /Delete> markers.

Table 5: **D**elete commands. A list of delete commands, their beginning and ending markers, and associated data.

Starting Marker	**Data**	**Ending Marker**
<Delete>	A group marker to delimit where data to delete a CRM record begins and ends. Only **D**elete commands are recognized in between these markers.	< /Delete>
<Comment>	A plain text message that is output to standard out when processed.	< /Comment>
<CRMID>	The identifier as returned from the Creation operation. CRMID is always plain text.	< /CRMID>
<GivenName>	Given name may be NULL, ASCII, or contain non-ASCII characters.	< /GivenName>
<PostCode>	Post code may be NULL, ASCII, or contain non-ASCII characters.	< /PostCode>
<Surname>	Family name may be NULL, ASCII, or contain non-ASCII characters.	< /Surname>
<Telephone>	Phone number may be NULL, ASCII, or contain non-ASCII characters.	< /Telephone>

4 Conclusion

The CSC-223 real-life application is to create, populate, and utilize a persistent client relationship management (**CRM**) system. The design of the CRM is to have a control file manage the creation, retrieval, update, and deletion (CRUD) database operations. Using a control file simplifies testing the CRM by ensuring consistency and reproduciability. This report details the specifications of the CRM control file, and how the different CRUD directives will be interpreted.

Sample Java source files that maybe of use in the creation of the CRM are included, as well as a sample control file are available in a GitHub repository.

A Sample Java listings

A collection of Java listings (see Table 6) that might be of use when designing your CRM implementation. The files are available on `GitHub`, and can be cloned to a local repository with this git command:

```
git clone git@github.com:Chuck-Cartledge/TCC-223-Control-file.git
```

or via a git graphical user interface (**GUI**). A complete sequence of steps to create a local git repository using a command line interface (**CLI**) is included (see Section B).

Table 6: Sample Java listings. These listings are provided as starting points for different aspects of your CRM implementation.

Listing	Explanation
A.1	The "classic" hello world program in Java.
A.2	Echo command line arguments to the screen.
A.3	How to read a string from a file to then process the line.
A.4	Base64 encoding enables <u>any</u> character set to be converted to ASCII.
A.5	Decoding a base64 string allows <u>any</u> character set to be safely passed using the ASCII character set.
A.6	Create a sample control file.
A.7	The sample control file created by Listing A.7. Note that some of the lines are too long to fit on the page, so you'll have to run the program to see the complete line.
A.8	How to persist Java data structures[1].

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Ubuntu!");
    }
}
```

Listing A.1: Classic: Hello World in Java.

```java
public class Echo {
    public static void main(String[] args) {
        System.out.println("Arguments received:");
        // Loop through all elements in the args array
        for (int i = 0; i < args.length; i++) {
            System.out.println("Argument " + (i + 1) + ": " + args[i]);
        }
    }
}
```

Listing A.2: Echo command line arguments to the screen.

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ReadFileBufferedReader {
    public static void main(String[] args) {
        String fileName = "sample.txt"; // Replace with your file path

        // Use try-with-resources to ensure the reader is closed
        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
            String line;
            // Read lines one by one until the end of the file (readLine() returns null)
            while ((line = reader.readLine()) != null) {
                System.out.println(line); // Process the line (e.g., print it)
            }
        } catch (IOException e) {
            // Handle exceptions such as FileNotFoundException or other I/O errors
            e.printStackTrace();
        }
    }
}
```

Listing A.3: Read lines from file, one at a time.

```java
import java.util.Base64;
import java.nio.charset.StandardCharsets;

public class Base64EncodeExample {
    public static void main(String[] args) {
        String originalString = "Hello, World!";

        // Convert the original string to bytes (UTF-8 is recommended)
        byte[] originalBytes = originalString.getBytes(StandardCharsets.
            UTF_8);

        // Get a Base64 Encoder instance
        Base64.Encoder encoder = Base64.getEncoder();

        // Encode the byte array to a Base64 string
        String encodedString = encoder.encodeToString(originalBytes);

        System.out.println("Original: " + originalString);
        System.out.println("Encoded: " + encodedString);
        // Output: SGVsbG8sIFdvcmxkIQ==
    }
}
```

Listing A.4: How to base64 encode a string.

```java
import java.util.Base64;
import java.nio.charset.StandardCharsets;

public class Base64DecoderExample {
    public static void main(String[] args) {
        // The Base64 encoded string
        String encodedString = "dGVzdCBpbnB1dA==";

        // 1. Get a Base64 Decoder instance
        Base64.Decoder decoder = Base64.getDecoder();

        // 2. Decode the string into a byte array
        byte[] decodedBytes = decoder.decode(encodedString);

        // 3. Convert the byte array back to the original string, specifying the character encoding (e.g., UTF-8)
        String originalString = new String(decodedBytes, StandardCharsets.UTF_8);

        System.out.println("Original: " + originalString);
        // Output: Original: test input
    }
}
```

Listing A.5: How to decode a base64 string.

```java
import java.util.Base64;
import java.nio.charset.StandardCharsets;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class CreateSampleControlFile {

    public static void writePlainTextStringArrayToFile(BufferedWriter writer, String startMarker, String[] lines, String endMarker)
    {

        try {
     writer.write(startMarker);
    writer.newLine();

    for (int i=0; i < lines.length; i += 3)
        {
      writer.write(lines[i]);
      writer.newLine();
      writer.write(lines[i+1]);
      writer.newLine();
      writer.write(lines[i+2]);
      writer.newLine();
        }
    writer.write(endMarker);
    writer.newLine();
      } catch (IOException e) {
            System.err.println("Error writing to file: " + e.getMessage());
            e.printStackTrace();
        }
    }

    public static void writeBase64EncodedStringArrayToFile(BufferedWriter writer, String startMarker, String[] lines, String endMarker)
    {
  Base64.Encoder encoder = Base64.getEncoder();
        byte[] originalBytes;
        String encodedString;

        try {
     writer.write(startMarker);
    writer.newLine();

    for (int i=0; i < lines.length; i += 3)
        {
```

```
      writer.write(lines[i]);
      writer.newLine();
      originalBytes = lines[i+1].getBytes(StandardCharsets.UTF_8);
      encodedString = encoder.encodeToString(originalBytes);
      writer.write(encodedString);
      writer.newLine();
      writer.write(lines[i+2]);
      writer.newLine();
        }
    writer.write(endMarker);
    writer.newLine();
      } catch (IOException e) {
            System.err.println("Error writing to file: " + e.getMessage())
                ;
            e.printStackTrace();
        }
    }


    public static void main(String[] args) {
        String [] systemLines1 = {
      "<DBFileName>",
      "CSC-223-CRM_database",
      "</DBFileName>",
      "<Save>",
      "",
      "</Save>",
      "<Exit>",
      "This line is written using CreateSampleControlFile.",
      "</Exit>"
  };

        String [] systemLines2 = {
      "<Output>",
      "CSC-223-CRM_database",
      "</Output>",
  };

  String [] createLines1 = {
      "<Surname>",
      "Cartledge",
      "</Surname>",
      "<GivenName>",
      "Charles",
      "</GivenName>",
      "<Telephone>",
      "+1-757-6332581",
      "</Telephone>",
      "<PostCode>",
```

```
        "23452",
        "</PostCode>"
    };
    String [] createLines2 = {
        "<GivenName>",
        "ball-bat",
        "</GivenName>",
        "<Telephone>",
        "+91 (22) 1234-5678",
        "</Telephone>",
        "<Surname>",
        "San Antonio",
        "</Surname>",
        "<PostCode>",
        "SW1A 1AA",
        "</PostCode>"
    };

    String [] retrieveLines = {
        "<CRMID>",
        "1",
        "</CRMID>",
        "<CRMID>",
        "Q2FydGxlZGdl",
        "</CRMID>"
    };

    String [] updateLines = {
        "<CRMID>",
        "Q2FydGxlZGdl",
        "</CRMID>",
        "<GivenName>",
        "Henry",
        "</GivenName>"
    };

    String [] deleteLines = {
        "<CRMID>",
        "Q2FydGxlZGdl",
        "</CRMID>",
        "<CRMID>",
        "1",
        "</CRMID>",
        "# A comment line",
        "  # yet another one.",
        "                 # this is the last one."
    };

    String [] longLines = {
```

```
      "<Sample>",
      "This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.  This is a short line that will be repeated many times to see what happens with a long line.",
      "</Sample>"
  };

        String fileName = "sampleControlFile.txt";


        // The try-with-resources statement automatically closes the writer, preventing resource leaks.
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName)))
      {
    writeBase64EncodedStringArrayToFile(writer, "<System>", systemLines1, "</System>");
    writeBase64EncodedStringArrayToFile(writer, "<Create>", createLines1, "</Create>");
    writeBase64EncodedStringArrayToFile(writer, "<Create>", createLines2, "</Create>");
    writePlainTextStringArrayToFile(writer,"<Retrieve>", retrieveLines, "</Retrieve>");
    writeBase64EncodedStringArrayToFile(writer, "<System>", systemLines2, "</System>");
    writeBase64EncodedStringArrayToFile(writer,"<Update>", updateLines, "</Update>");
    writePlainTextStringArrayToFile(writer,"<Delete>", deleteLines, "</Delete>");
    writeBase64EncodedStringArrayToFile(writer,"<Ignore>", longLines, "</Ignore>");
    System.out.println("Successfully wrote to the file.");

      } catch (IOException e) {
            System.err.println("Error writing to file: " + e.getMessage());
            e.printStackTrace();
```

```
            }
        }
}
```

Listing A.6: Create a sample control file.

```
<System>
<DBFileName>
Q1NDLTIyMy1DUk1fZGF0YWJhc2U=
</DBFileName>
<Save>

</Save>
<Exit>
VGhpcyBsaW5lIGlzIHdyaXR0ZW4gdXNpbmcgQ3JlYXRlU2FtcGxlQ29udHJvbEZpbGUu
</Exit>
</System>
<Create>
<Surname>
Q2FydGxlZGdl
</Surname>
<GivenName>
Q2hhcmxlcw==
</GivenName>
<Telephone>
KzEtNzU3LTYzMzI1ODE=
</Telephone>
<PostCode>
MjM0NTI=
</PostCode>
</Create>
<Create>
<GivenName>
YmFsbC1iYXQ=
</GivenName>
<Telephone>
KzkxICgyMikgMTIzNC01Njc4
</Telephone>
<Surname>
U2FuIEFudG9uaW8=
</Surname>
<PostCode>
U1cxQSAxQUE=
</PostCode>
</Create>
<Retrieve>
<CRMID>
1
</CRMID>
<CRMID>
Q2FydGxlZGdl
</CRMID>
</Retrieve>
<System>
```

```
<Output>
Q1NDLTIyMy1DUk1fZGF0YWJhc2U=
</Output>
</System>
<Update>
<CRMID>
UTJGeWRHeGxaR2Rs
</CRMID>
<GivenName>
SGVucnk=
</GivenName>
</Update>
<Delete>
<CRMID>
Q2FydGxlZGdl
</CRMID>
<CRMID>
1
</CRMID>
# A comment line
  # yet another one.
                  # this is the last one.
</Delete>
<Ignore>
<Sample>
VGhpcyBpcyBhIHNob3J0IGxpbmUgdGhhdCB3aWxsIGJlIHJlcGVhdGVkIG1hbnkgdGltZXMgdG8gc2VlIHdoYX
    ==
</Sample>
</Ignore>
```

Listing A.7: A sample control file.

```java
import java.io.*;

class Demo implements Serializable {
    public int a;
    public String b;

    public Demo(int a, String b) {
        this.a = a;
        this.b = b;
    }
}

public class Geeks {
    public static void main(String[] args) {
        Demo object = new Demo(1, "geeksforgeeks");
        String filename = "file.ser";

        // Serialization
        try {
            FileOutputStream file = new FileOutputStream(filename);
            ObjectOutputStream out = new ObjectOutputStream(file);
            out.writeObject(object);
            out.close();
            file.close();
            System.out.println("Object has been serialized");

        } catch (IOException ex) {
            System.out.println("IOException is caught");
        }

        Demo object1 = null;

        // Deserialization
        try {
            FileInputStream file = new FileInputStream(filename);
            ObjectInputStream in = new ObjectInputStream(file);
            object1 = (Demo) in.readObject();
            in.close();
            file.close();
            System.out.println("Object has been deserialized");
            System.out.println("a = " + object1.a);
            System.out.println("b = " + object1.b);

        } catch (IOException ex) {
            System.out.println("IOException is caught");
        } catch (ClassNotFoundException ex) {
            System.out.println("ClassNotFoundException is caught");
        }
```

```
        }
}
```

Listing A.8: Persisting Java data.

B High Level GitHub Instructions

A quote from GitHub:

> *"Git is a version control system that intelligently tracks changes in files. Git is particularly useful when you and a group of people are all making changes to the same files at the same time.*
> *Typically, to do this in a Git-based workflow, you would:*
> *– **Create a branch** off from the main copy of files that you (and your collaborators) are working on.*
> *– **Make edits** to the files independently and safely on your own personal branch.*
> *– Let Git intelligently **merge** your specific changes back into the main copy of files, so that your changes don't impact other people's updates.*
> *– Let Git **keep track** of your and other people's changes, so you all stay working on the most up-to-date version of the project."*
>
> GitHub Staff [2]

If a GitHub repository exists, the owner will normally send an e-mail invitation for you to create a free account on GitHub.

Once you have created your free account, then you will need to perform the following operations. The exact commands and menu options to be used will vary based on which GUI, IDE, or CLI you use.

1. Create a local directory where all your code and files will live.
   (A sample command could be: `mkdir /tmp/Temp/`)

2. Change to your local directory.
   (A sample command could be: `cd /tmp/Temp`)

3. Clone the starting repository from GitHub to your local directory
   (A sample command could be:
   `git clone git@github.com:Chuck-Cartledge/TCC-223-Control-file.git`
)

4. Change directory to the repository you've just cloned from GitHub.
   (A sample command could be: `cd /tmp/Temp/TCC-223-Control-file/`)

5. Create a local branch for you to do your work in.
   (A sample command could be: `git checkout -b YourName`)
   Note: YourName is your personal name, not the string YourName

6. Make any changes that you want to the files in your branch. This might take a while because you'll make changes and test your changes many times until you have a collection of files you want to save to GitHub.

7. Come up with a name that describes the changes you have done and how you can refer to them in the future. Tell `git` what that name is.
   (A sample command could be: `git commit -am ‘‘First go around’’`)

8. Push that collection of changed files to GitHub.
   (A sample command could be: `git push -u origin YourName`)
   Note: YourName needs to be replaced by the branch name you created

References

[1] GeeksForGeeks Staff, Serialization and Deserialization in Java, https://www.geeksforgeeks.org/java/serialization-and-deserialization-in-java/, 2025.

[2] GitHub Staff, About GitHub and Git, https://docs.github.com/en/get-started/start-your-journey/about-github-and-git, 2026.

[3] Wikipedia Staff, Base64, https://en.wikipedia.org/wiki/Base64, 2026.